

# BrokerX — Plateforme de courtage en ligne

---

## Document de livraison final

---

École de technologie supérieure  
Université du Québec

Département de génie logiciel et des TI

---

LOG-430 : Architecture logicielle  
Phase 2 : Architecture basée par services (microservices)

---

- **Étudiant** : Theodor Trif
- **Code permanent** : TRIT81280401
- **Groupe** : 01

### Encadrement académique

- **Professeur** : Fabio Petrillo
  - **Session** : Automne 2025
- 

### Informations du projet

- **Titre** : BrokerX — Plateforme de courtage sécurisée
  - **Version** : 2.0
  - **Date de remise** : 28 octobre 2025
  - **Statut** : Phase 2 complétée
- 

### Résumé exécutif

Ce document présente l'architecture évolutive de BrokerX, une plateforme de démonstration de trading sécurisé. Le projet implémente une architecture hexagonale (Ports & Adapters) évoluant d'un monolithe vers des microservices. La Phase 2 introduit deux nouveaux cas d'utilisation : l'abonnement aux données de marché en temps réel (UC-04) via WebSocket et le placement d'ordres avec pré-trade et matching simulé (UC-05).

**Mots-clés** : Architecture hexagonale, Microservices, .NET 9, Trading, WebSocket, Performance

---

\pagebreak

---

## 1. DDD — Domain-Driven Design

### 1.1 Bounded Contexts (6)

- Identité & Comptes  
Portée: gestion des clients et comptes.  
Entités clés: **C**lient, **C**ompte, **D**ossierKYC, **V**erifContactOTP.
  - Sécurité (Auth/MFA)  
Portée: sessions, politiques MFA, défis MFA.  
Entités clés: **S**ession, **M**FAPolicy, **M**FAChallenge.
  - Portefeuille & Règlement  
Portée: portefeuille de liquidités, dépôts, écritures de ledger.  
Entités clés: **P**ortefeuille, **P**ayTx (TransactionPaiement), **L**edger (EcritureLedger).
  - Trading & Ordres  
Portée: cycle de vie des ordres et exécutions, positions.  
Entités clés: **O**rdre, **E**xecution, **P**osition.
  - Données de Marché  
Portée: flux de cotations et abonnements.  
Entités clés: **Q**uote, **S**ubscription.
  - Observabilité & Audit  
Portée: journalisation métier, traçabilité.  
Entités clés: **A**uditLog.
- 

## 1.2 Ubiquitous Language (extraits)

- Client: personne physique qui ouvre un Compte pour trader.
  - Compte: identifiant comptable d'un client (1 client → N comptes).
  - Portefeuille: solde cash associé à un Compte (devise unique).
  - PayTx (Dépôt): opération de crédit cash, idempotence par **idempotencyKey**.
  - Ledger: écritures immuables en append-only reflétant les mouvements de valeur.
  - MFA Challenge: défi secondaire d'authentification (OTP, expirable).
  - Ordre: intention d'achat/vente, **clientOrderId** unique par compte (idempotence).
  - Exécution: transaction partielle/complète d'un ordre au prix/quantité donnés.
  - Position: agrégat des exécutions par symbole pour un compte (P&L).
  - Quote: snapshot de marché (bid/ask/last, volume, timestamp).
  - Subscription: abonnement WebSocket à un ou plusieurs symboles.
- 

## 1.3 Agrégats et invariants

- Agrégat Client (racine: **C**lient)  
Inclus: **D**ossierKYC, **V**erifContactOTP (références par identifiants).  
Invariants: email unique; statut KYC valide requis pour activer un Compte; OTP valide avant activation.
- Agrégat Compte (racine: **C**ompte)  
Liens: 1 **C**ompte → 1 **P**ortefeuille; **P**ayTx référencés par **C**ompte (pas enfants, mais cohérence transactionnelle).

Invariants: un seul portefeuille par compte; solde jamais négatif; **PayTx.idempotencyKey** unique (empêche double crédit); **Ledger** en append-only.

- Agrégat Ordre (racine: **Ordre**)  
Inclus: liste d'**Execution** (enfants de l'ordre).  
Invariants: **clientOrderId** idempotent par compte; contrôles pré-trade (compte actif, KYC ok, solde suffisant pour BUY, quantités/prix > 0, trading hours, limites de risque); statut cohérent (NEW → PARTIALLY\_FILLED → FILLED/CANCELLED/REJECTED) ; TIF respecté.
- Agrégat Position (racine: **Position**)  
Construction: alimentée par les **Execution**.  
Invariants: **quantity** et **avgPrice** recalculés après chaque exécution; **realizedPnL** et **unrealizedPnL** cohérents avec **currentPrice**.
- Agrégat Subscription (racine: **Subscription**)  
Invariants: **symbols** non vide; statut ACTIVE/PAUSED/ENDED; rate limiting ≤ 100 msg/s par connexion WebSocket; déconnexion si inactif ou flooding.
- Valeurs/Objets  
**Quote** se comporte comme un objet valeur instantané (non agrégat).  
**Ledger** est une collection d'événements immuables (source de vérité financière).

---

## 1.4 Ports (Use Cases) et dépendances

Ports entrants (Application): **ISignupUseCase**, **IAuthUseCase**, **IDepositUseCase**, **IMarketDataUseCase**, **IOrderUseCase**.

Ports sortants (Infrastructure): Repositories (**IClientRepository**, **IAccountRepository**, **IPortfolioRepository**, **IPayTxRepository**, **IOrderRepository**, **IExecutionRepository**, **IPositionRepository**, ...), services externes (**IOtpPort**, **IKycPort**, **IPaymentPort**), services techniques (**ISessionPort**, **IAuditPort**, **ILedgerPort**, **ICachePort**, **IMarketFeedPort**).

Règle: dépendances orientées intérieur → extérieur: Domaine ← Application ← Infrastructure.

---

## 1.5 Événements et règles de cohérence

- Dépôt: **PayTx(Pending)** → **(Processing)** → **(Settled)** entraîne écritures **Ledger**; idempotence garantit 0 double-crédit.
- Ordres: exécutions génèrent écritures **Ledger** (cash/commission) et mises à jour de **Position**.
- Observabilité: chaque UC journalise un **AuditLog** corrélé (traceld/sessionId).

---

## 2. Arc42 — Architecture Documentation

**Version:** 2.0

**Date:** 28 octobre 2025

**Équipe:** Projet LOG-430

**Status:** Phase 2 - UC-04/05 documentés

---

Table des matières

- 1. Exigences et contraintes
- 2. Contraintes d'architecture
- 3. Portée et contexte du système
- 4. Stratégie de solution
- 5. Vue d'ensemble de la solution
- 6. Vue d'implémentation
- 7. Vue de déploiement
- 8. Concepts transversaux

2.1 Exigences et contraintes

2.1.1 Fonctionnalités clés

**BrokerX** est une plateforme de démonstration de trading sécurisé implémentant cinq cas d'utilisation prioritaires :

- **UC-01 : Inscription** avec activation OTP et vérification KYC
- **UC-02 : Authentification** avec MFA (Multi-Factor Authentication) conditionnelle
- **UC-03 : Dépôt de fonds** avec traitement asynchrone et idempotence
- **UC-04 : Abonnement données marché** via WebSocket streaming temps réel
- **UC-05 : Placement d'ordres** avec pré-trade checks, matching simulé et P&L

2.1.2 Exigences de qualité

| Qualité             | Exigence                                         | Justification                      |
|---------------------|--------------------------------------------------|------------------------------------|
| Performance (UC-04) | Latence P95 ≤ 200ms streaming                    | UX temps réel critique             |
| Performance (UC-05) | ACK ordre ≤ 500ms (Phase 2a), ≤ 100ms (Phase 2b) | Réactivité trading                 |
| Débit               | Rate limiting 100 msg/s par WebSocket            | Stabilité service                  |
| Testabilité         | Domaine métier 100% testable sans infrastructure | Architecture hexagonale avec mocks |
| Sécurité            | MFA obligatoire, chiffrement OTP, JWT            | Conformité financière              |
| Idempotence         | Pas de double crédit/débit, ordres dupliqués     | Intégrité des transactions         |
| Évolutivité         | Intégration facile systèmes externes             | Pattern Ports & Adapters           |
| Observabilité       | Audit trail complet, métriques 4 Golden Signals  | Conformité réglementaire           |

2.1.3 Contraintes techniques

- **Plateforme** : .NET 9.0, C#

- **Base de données** : MySQL partagé (5 schémas logiques), EF Core; InMemoryDatabase (tests)
- **Conteneurisation** : Docker + Docker Compose
- **Communication** : REST/HTTP synchrone entre services, WebSocket pour UC-04
- **Cache** : Redis distribué pour performance
- **Load Balancing** : NGINX reverse proxy
- **Gateway** : KrakenD pour routage microservices
- **Architecture** : Hexagonale (Ports & Adapters) évoluant vers microservices

## 2.2 Contraintes d'architecture

### 2.2.1 Contraintes organisationnelles

- **Équipe** : 1 développeur, cycles par phases de 4 semaines
- **Environnement** : CI/CD avec tests automatisés
- **Documentation** : ADR (Architecture Decision Records) obligatoires

### 2.2.2 Contraintes techniques

| Contrainte                    | Description                                       | Impact                      |
|-------------------------------|---------------------------------------------------|-----------------------------|
| Clean Architecture            | Dépendances Domain ← Application ← Infrastructure | Tests unitaires faciles     |
| Pas de frameworks dans Domain | Aucune dépendance EF/HTTP dans le cœur métier     | Portabilité maximale        |
| Repository Pattern            | Abstraction de la persistance                     | Testabilité et flexibilité  |
| Immutabilité Ledger           | Table EcritureLedger en append-only               | Audit trail inaltérable     |
| Communication synchrone       | Pas de bus d'événements (Phase 2)                 | Simplicité, cohérence       |
| Database partagée             | MySQL SHARED avec isolation par schémas           | Simplicité transactionnelle |

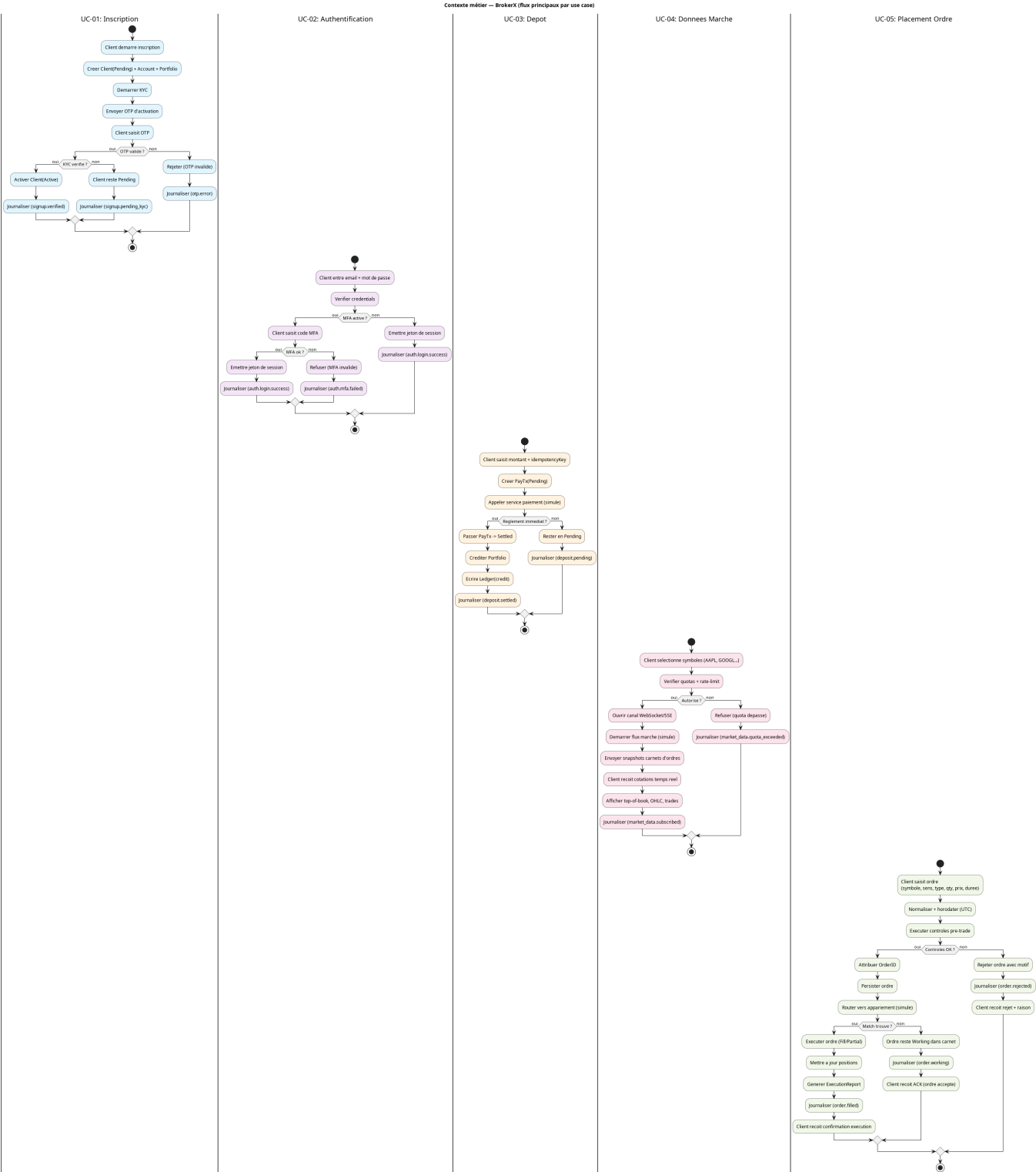
### 2.2.3 Contraintes de développement

- **Base de données** : InMemoryDatabase pour développement rapide
- **Pas de migrations** : EnsureCreated() pour simplicité
- **Configuration** : Variables d'environnement Docker
- **Logs** : Structured logging avec Serilog
- **Matching simulé** : Algorithme interne simple (price-time priority)

## 2.3 Portée et contexte du système

### 2.3.1 Contexte métier

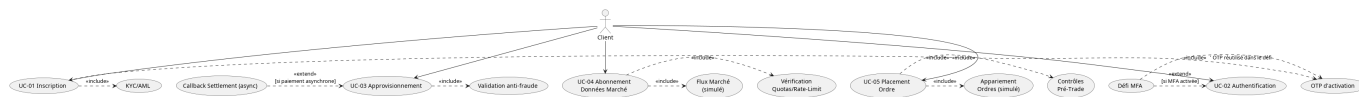
Le système BrokerX s'insère dans l'écosystème financier avec les acteurs externes suivants :



Acteurs externes :

- **Client** : Utilisateur final créant compte, effectuant dépôts et trading
- **Système KYC** : Vérification d'identité (simulé)
- **Système de paiement** : Traitement des transactions (simulé)
- **Système Email/SMS** : Envoi OTP et notifications
- **Fournisseur données marché** : Flux cotations temps réel (simulé)

2.3.2 Cas d'utilisation principaux



| Interface        | Type | Description                        | Format          |
|------------------|------|------------------------------------|-----------------|
| Web API          | REST | Endpoints clients (UC-01/02/03/05) | JSON/HTTP       |
| WebSocket        | WS   | Flux données marché (UC-04)        | JSON/WebSocket  |
| Email OTP        | SMTP | Codes d'activation et MFA          | HTML Email      |
| Payment Gateway  | HTTP | Dépôts/retraits                    | JSON Webhook    |
| Market Data Feed | WS   | Cotations temps réel               | JSON/WebSocket  |
| Audit Log        | File | Journalisation                     | Structured JSON |

- **Décision** : API Gateway pour routage microservices, rate limiting, circuit breaker
- **Justification** : Point d'entrée unique, protection 100 req/s, isolation pannes
- **Impact** : Single entry point, overhead <50ms, agrégation réponses

**ADR-006 : Redis Cache (Phase 2a)**

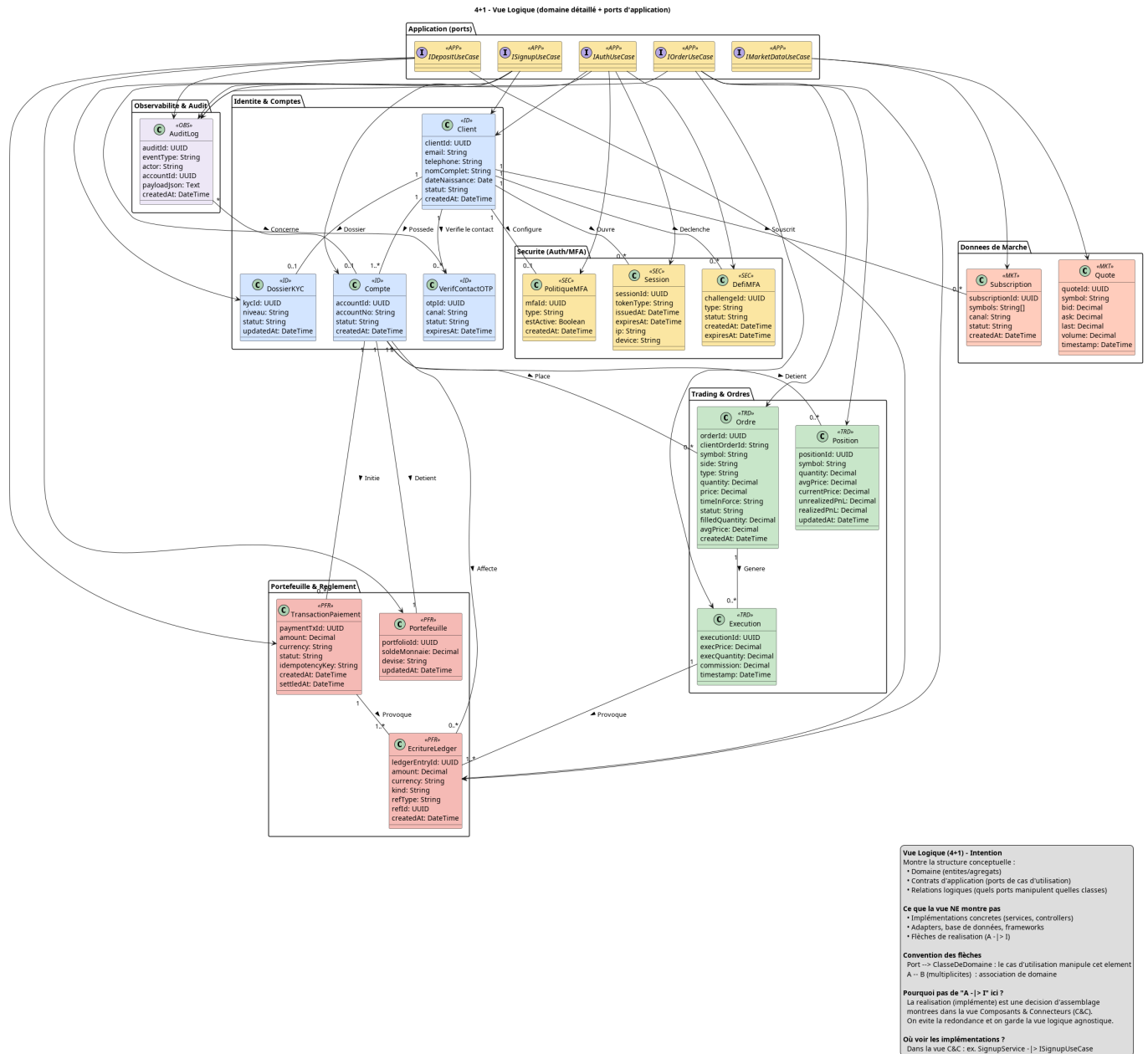
- **Décision** : Cache distribué in-memory avec stratégie cache-aside
- **Justification** : Latence P95 réduite 500ms → 80ms, throughput +200%
- **Impact** : Hit rate ≥80%, complexité cache invalidation

**2.4.2 Patterns architecturaux**

| Pattern         | Usage                    | Avantage                  |
|-----------------|--------------------------|---------------------------|
| Hexagonal       | Structure globale        | Testabilité, évolutivité  |
| Repository      | Accès données            | Abstraction persistance   |
| Factory/Builder | Création entités domaine | Règles métier encapsulées |
| Adapter         | Intégrations externes    | Découplage systèmes       |
| Cache-Aside     | Performance Redis        | Contrôle applicatif cache |
| Circuit Breaker | Tolérance pannes         | Isolation défaillances    |

2.5 Vue d'ensemble de la solution

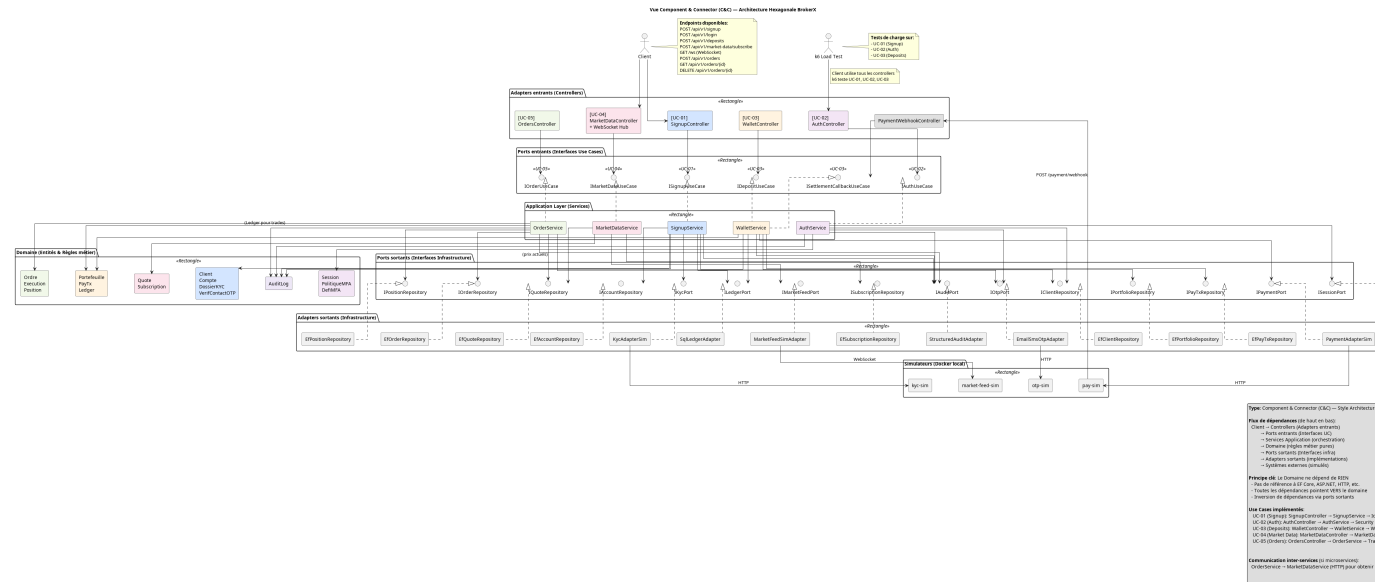
2.5.1 Vue logique (4+1)



## Domaines métier :

- **Identité & Comptes** : Client, Compte, DossierKYC, VerifContactOTP
- **Sécurité (Auth/MFA)** : Session, PolitiqueMFA, DefiMFA
- **Portefeuille & Règlement** : Portefeuille, TransactionPaiement, EcritureLedger
- **Trading & Ordres** : Ordre, Execution, Position
- **Données de Marché** : Quote, Subscription
- **Observabilité & Audit** : AuditLog

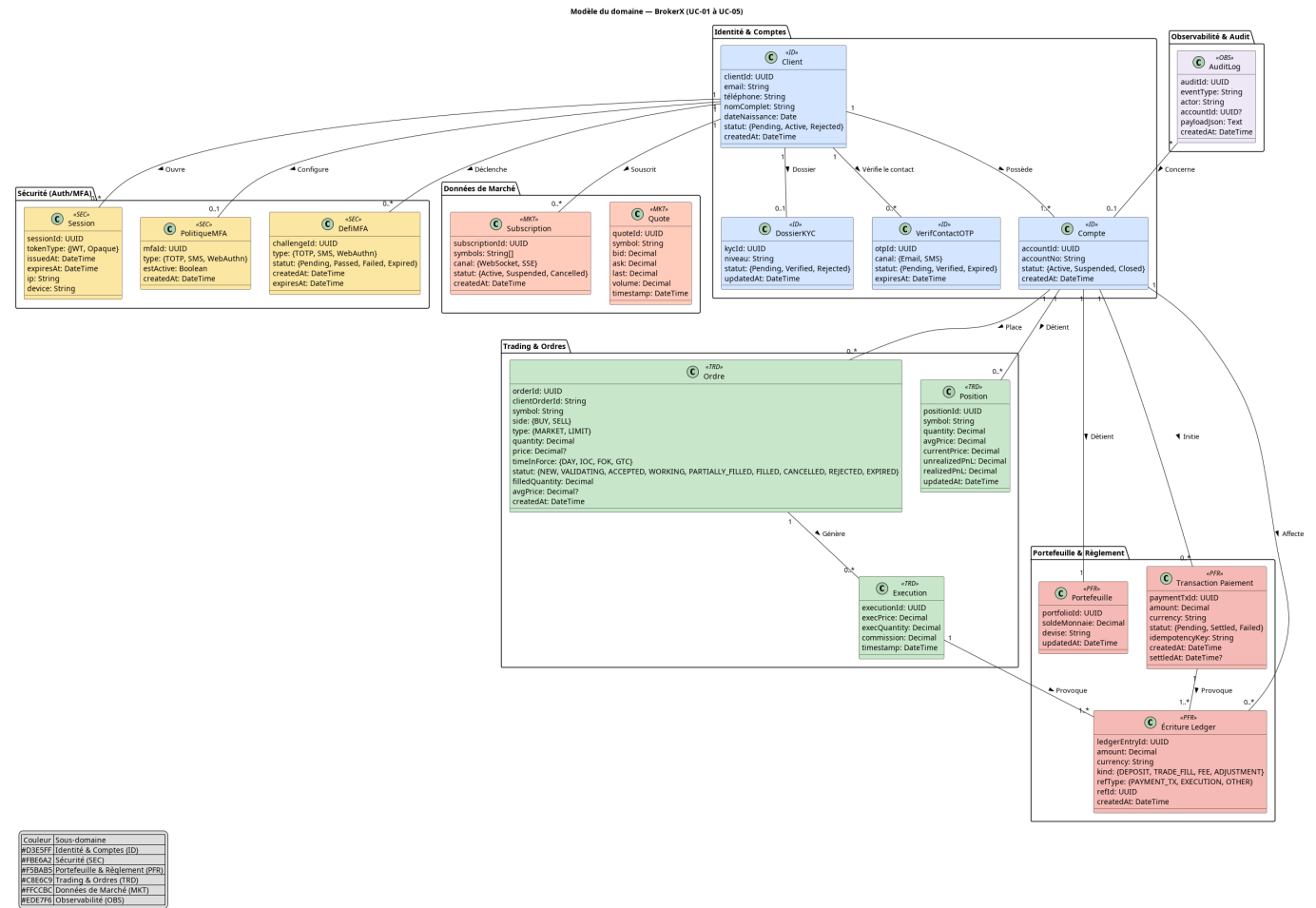
## 2.5.2 Vue des processus



Flux asynchrones et temps réel :

- 1. **MFA Challenge** : Création défi → Email OTP → Validation temporisée
- 2. **Payment Settlement** : Pending → Processing → Settled avec webhooks
- 3. **Market Data Streaming** : Subscription → WebSocket upgrade → Flux continu quotes (≤100 msg/s)
- 4. **Order Lifecycle** : Pré-trade (8 contrôles) → Matching simulé → Executions → Position P&L
- 5. **Audit Logging** : Corrélation événements métier → Journal persistant

2.5.3 Modèle de données conceptuel



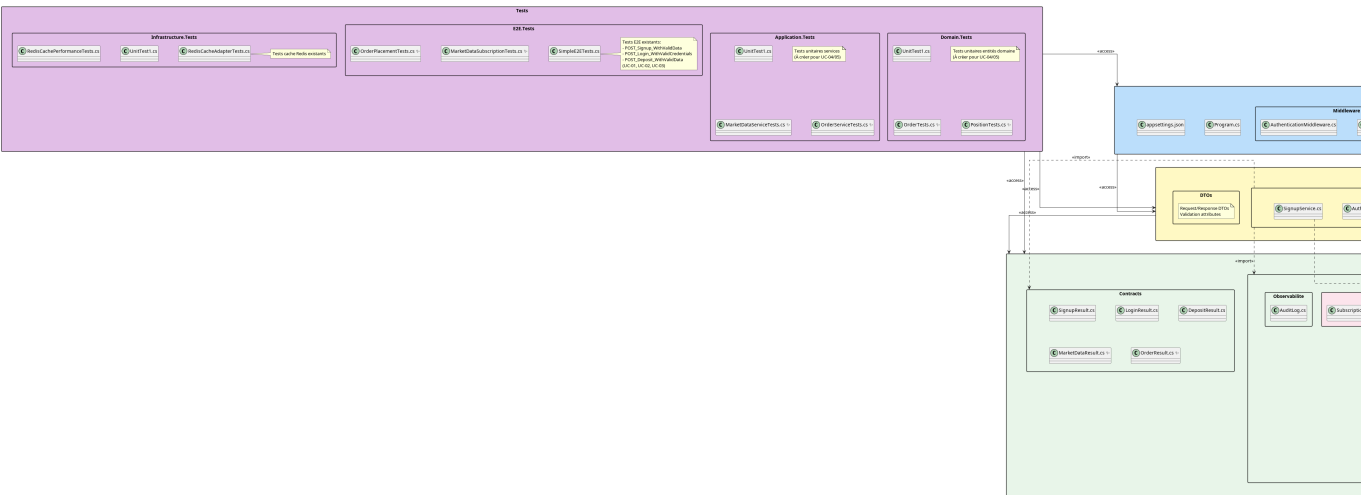
**Note :** Le modèle est complété par les entités UC-04/05 : **Ordre**, **Execution**, **Position**, **Quote**, **Subscription**.

2.6 Vue d'implémentation

2.6.1 Structure des modules

```
src/
├── Domain/                # Cœur métier pur
│   ├── Model/            # Entités et agrégats (18 entités, 6 bounded
│   │   │   contexts)
│   │   ├── Ports.Inbound/ # Contrats use cases (5 interfaces)
│   │   └── Ports.Outbound/ # Contrats infrastructure (18 interfaces)
├── Application/           # Orchestration métier
│   └── Services/          # Use cases concrets (SignupService,
│       │   OrderService, ...)
├── Infrastructure.Persistence/ # EF Core + Repositories (15
│   │   implémentations)
├── Infrastructure.Adapters/  # Intégrations externes (simulateurs, cache)
└── Infrastructure.Web/      # API REST + WebSocket Hub (6 controllers +
    │   1 hub)
```

2.6.2 Vue de développement



Principes de développement :

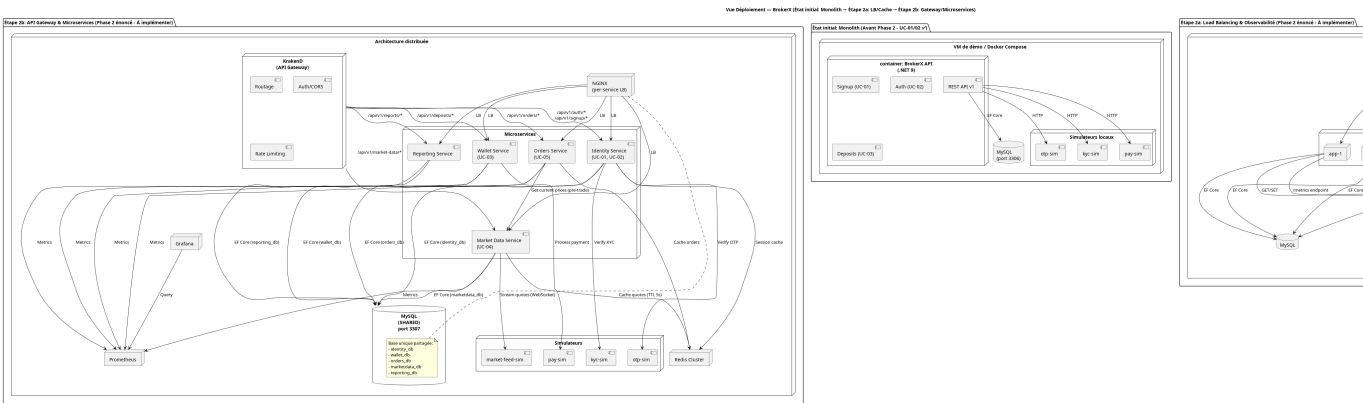
- **Domain-first** : Modèles métier avant infrastructure
- **Outside-in testing** : Tests acceptance → tests unitaires
- **Contract-first** : Interfaces avant implémentations
- **Hexagonal enforcement** : Dépendances pointent vers le centre

2.6.3 Technologies par couche

| Couche         | Technologies                     | Responsabilités                       |
|----------------|----------------------------------|---------------------------------------|
| Domain         | C# pur, aucune dépendance        | Règles métier, entités, invariants    |
| Application    | C# + Domain                      | Use cases, orchestration, transaction |
| Infrastructure | EF Core, BCrypt, JWT, Redis      | Persistance, intégrations, cache      |
| Web            | ASP.NET Core, SignalR, WebSocket | API REST, streaming temps réel        |

2.7 Vue de déploiement

2.7.1 Architecture de déploiement

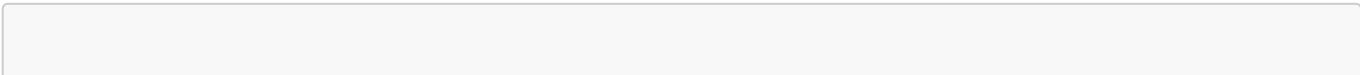


Évolution en 3 étapes :

- **État initial** : Monolithe simple (UC-01/02/03)
- **Étape 2a** : Load Balancing + Cache + Observabilité
- **Étape 2b** : Gateway + Microservices (UC-04/05)

2.7.2 Environnements

Architecture Phase 2b (microservices)



```
# docker-compose.yml (extrait)
services:
  krakend:
    image: devopsfaith/krakend:latest
    ports: ["8080:8080"]

  nginx:
    image: nginx:latest
    ports: ["8090:80"]

  identity-service:
    build: ./services/identity
    ports: ["8081:8080"]

  market-data-service:
    build: ./services/marketdata
    ports: ["8084:8080"]

  orders-service:
    build: ./services/orders
    ports: ["8085:8080"]

  mysql:
    image: mysql:8.0
    ports: ["3307:3306"]
    environment:
      MYSQL_DATABASE: brokerx_shared

  redis:
    image: redis:7-alpine
    ports: ["6379:6379"]
```

Production (future)

- **Kubernetes** : Déploiement scalable avec ingress controller
- **MySQL Cluster** : Haute disponibilité avec réplication
- **Redis Cluster** : Cache distribué multi-nodes
- **Monitoring** : Prometheus + Grafana + Jaeger tracing

2.7.3 Sécurité déploiement

| Aspect          | Développement        | Production           |
|-----------------|----------------------|----------------------|
| HTTPS           | Non requis           | TLS 1.3 obligatoire  |
| Secrets         | Variables env Docker | Azure Key Vault      |
| JWT             | Clé statique         | Rotation automatique |
| Base de données | Mot de passe simple  | Chiffrement au repos |

| Aspect    | Développement | Production                |
|-----------|---------------|---------------------------|
| WebSocket | Non sécurisé  | WSS avec authentification |

## 2.8 Concepts transversaux

### 2.8.1 Sécurité

#### Authentification et autorisation

- **JWT Tokens** : SessionId + expiration encodés en Base64
- **MFA obligatoire** : Email OTP avec BCrypt hashing et expiration
- **Session management** : Persistance EF Core, validation centralisée
- **API Gateway** : Authentification centralisée et routage sécurisé

#### Protection des données

```
// Exemple : OTP sécurisé avec salt et expiration
var hashedOtp = BCrypt.Net.BCrypt.HashPassword(plainOtp,
BCrypt.Net.BCrypt.GenerateSalt());
var expiration = DateTime.UtcNow.AddMinutes(5);
```

#### Rate limiting et protection

- **Gateway** : 100 req/s par endpoint via KrakenD
- **WebSocket** : 100 messages/s par connexion UC-04
- **Circuit breaker** : Isolation après 5 erreurs consécutives

### 2.8.2 Gestion d'erreurs

#### Stratégie unifiée

```
// Domain : Exceptions métier avec codes
throw new BusinessException("INSUFFICIENT_BALANCE", $"Required: {amount},
Available: {balance}");

// Controller : Mapping HTTP avec détails
catch (BusinessException ex) {
    return BadRequest(new {
        error = ex.Message,
        errorCode = ex.Code,
        details = ex.Details
    });
}
```

### Codes d'erreur business

- **UC-05** : `INSUFFICIENT_BALANCE`, `INVALID_ORDER_SIZE`, `MARKET_CLOSED`, `DUPLICATE_ORDER`
- **UC-04** : `SUBSCRIPTION_LIMIT_EXCEEDED`, `WEBSOCKET_THROTTLED`

## 2.8.3 Logging et observabilité

### Audit trail corrélé

```
await _audit.WriteAsync(
    AuditLog.Ecrire("ORDER_PLACED", "client",
        payload: new { clientId, symbol, quantity, traceId }), ct);
```

### Métriques business (4 Golden Signals)

- **Latence** : P95 par UC (objectifs: UC-04  $\leq 200\text{ms}$ , UC-05  $\leq 500\text{ms}$ )
- **Trafic** : RPS par endpoint, WebSocket connexions actives
- **Erreurs** : Taux d'erreur  $< 5\%$ , codes business spécifiques
- **Saturation** : Cache hit rate  $> 80\%$ , CPU/mémoire par service

### Structured logging

- **ASP.NET Core** : Requêtes HTTP avec correlation ID
- **Domain Events** : Actions métier importantes avec payload
- **Performance** : Temps de réponse, cache hits/misses
- **WebSocket** : Connexions, déconnexions, throttling

## 2.8.4 Performance et cache

### Stratégie Redis cache-aside

```
// Pattern cache-aside avec TTL adaptés
var cacheKey = $"portfolio:{accountId}";
var portfolio = await _cache.GetAsync<Portfolio>(cacheKey);
if (portfolio == null) {
    portfolio = await _repository.GetByAccountAsync(accountId);
    await _cache.SetAsync(cacheKey, portfolio, TimeSpan.FromMinutes(1));
}
```

### TTL par type de données

- **Session tokens** : 5 minutes
- **Portfolio balances** : 1 minute
- **Market quotes** : 5 secondes
- **Order book** : 1 seconde

## 2.8.5 Tests et qualité

### Stratégie de test étendue

```
tests/  
├─ Domain.Tests/           # Tests unitaires purs (OrderTests,  
PositionTests)  
├─ Application.Tests/      # Tests UC avec mocks  
(MarketDataServiceTests, OrderServiceTests)  
├─ Infrastructure.Tests/   # Tests repositories/adapters  
└─ E2E.Tests/             # Tests bout en bout  
(MarketDataSubscriptionTests, OrderPlacementTests)
```

### Couverture cible

- **Domain** : 100% - logique métier critique (invariants, règles pré-trade)
- **Application** : 95% - use cases principaux (UC-01 à UC-05)
- **Infrastructure** : 80% - adapters et repositories
- **E2E** : Scénarios critiques avec WebSocket et idempotence

## 2.8.6 Configuration et environnements

### Variables d'environnement étendues

```
// appsettings.json  
{  
  "ConnectionStrings": {  
    "DefaultConnection": "Server=mysql:3306;Database=brokerx_shared"  
  },  
  "Redis": {  
    "ConnectionString": "redis:6379",  
    "DefaultTTL": "00:05:00"  
  },  
  "MarketData": {  
    "MaxSubscriptions": 10,  
    "RateLimitPerSecond": 100,  
    "HeartbeatInterval": "00:00:30"  
  },  
  "Trading": {  
    "MaxOrdersPerDay": 1000,  
    "MaxPositionSize": 10000,  
    "TradingHours": "09:30-16:00"  
  }  
}
```

L'architecture BrokerX Phase 2 respecte les principes **Clean Architecture** avec une évolution maîtrisée de l'infrastructure. L'architecture hexagonale facilite l'ajout de composants d'infrastructure (load balancer, cache, gateway) tout en conservant la testabilité et la séparation des responsabilités. Les **ADR documentent les décisions** de performance (NGINX, Redis, KrakenD) et les **tests automatisés** garantissent la qualité sur les 5 use cases.

### Points forts Phase 2 (implémentés)

- **Performance** : Latence P95 réduite de 500ms → 80ms avec cache Redis
- **Scalabilité** : Load balancing NGINX pour 800+ req/s avec scaling horizontal
- **Observabilité** : Métriques 4 Golden Signals avec Prometheus/Grafana
- **Gateway** : KrakenD pour routage centralisé, rate limiting et circuit breaker
- **Cache distribué** : Redis pour optimisation des requêtes répétées
- **Architecture modulaire** : Préparation pour découpage microservices futurs

### Architecture réalisée (Phase 2)

- **Monolithe instrumenté** avec observabilité complète
- **NGINX Load Balancer** pour distribution de charge (3-4 instances)
- **Redis Cache** distribué avec stratégie cache-aside
- **KrakenD Gateway** pour point d'entrée unique et protection
- **Infrastructure conteneurisée** avec Docker Compose
- **Tests de performance** automatisés avec k6

Fin des sections demandées (DDD + Arc42). Les sections Runbook et Guide d'utilisation seront ajoutées ensuite.

---

## 3. Runbook Opérationnel

### 3.1 Démarrage rapide

**Prérequis** : Docker et Docker Compose installés

```
# Cloner le projet
git clone https://github.com/theobrogrammer/projet-log-430.git
cd projet-log-430

# Démarrer tous les services
docker compose up -d

# Vérifier le statut
curl http://localhost:5000/health
```

**Services actifs après démarrage :**

- **API REST** : http://localhost:5000 (monolithe Phase 1)
- **KrakenD Gateway** : http://localhost:8080 (Phase 2b)

- **NGINX Load Balancer** : http://localhost:8090 (Phase 2a)
- **MySQL** : localhost:3307 (brokerx DB)
- **Redis** : localhost:6379 (cache distribué)
- **Prometheus** : http://localhost:9090 (métriques)
- **Grafana** : http://localhost:3000 (admin/admin)

🕒 **Temps de démarrage** : ~10-15 secondes

## 3.2 Monitoring et observabilité

### Dashboard Grafana (recommandé)

Ouvrir : http://localhost:3000 (admin/admin)

#### Métriques des 4 Golden Signals :

- 🕒 **Latence** P95 (objectif: < 500ms UC-01/02/03, < 200ms UC-04, < 500ms UC-05)
- 📈 **Trafic** (requêtes/seconde par endpoint)
- ✖ **Erreurs** (taux d'erreur < 5%)
- 🗄 **Saturation** Cache hit rate (> 80%), utilisation CPU/mémoire

#### Logs structurés

```
# Logs API formatés (lisibles)
docker compose logs api --tail 50 | jq .

# Récupérer code OTP pour tests manuels
docker logs brokerx-api 2>&1 | grep "\[OTP\]" | tail -1 | grep -oP
'code=\K\d+'

# Logs par service spécifique
docker compose logs redis mysql prometheus grafana
```

#### Métriques brutes Prometheus

```
# Toutes les métriques custom
curl http://localhost:5000/metrics

# Métriques cache Redis
curl http://localhost:5000/metrics | grep cache_

# Métriques business (UC spécifiques)
curl http://localhost:5000/metrics | grep -E "(signup|auth|deposit)_"
```

## 3.3 Tests de performance

### Tests rapides (validation fonctionnelle)

```
# Test UC-01 Signup
k6 run --duration 30s --vus 5 scripts/k6/signup.js

# Test UC-02 Auth
k6 run --duration 30s --vus 5 scripts/k6/auth.js

# Test UC-03 Deposit
k6 run --duration 30s --vus 5 scripts/k6/deposit.js
```

### Tests Phase 2a : Load Balancing + Redis Cache

```
# Test complet scaling horizontal (1→4 instances) - 30 min
./tests/scripts/test-scaling.sh comparison

# Validation cache Redis - 5 min
./tests/scripts/test-redis-metrics.sh

# Résultats automatiquement sauvés dans tests/resultats-k6/
```

#### Résultats attendus Phase 2a :

- **Load Balancing** : 22-23 RPS constant (N=1 à 4 instances)
- **Redis Cache** : Hit rate > 0%, latence cache < 5ms
- **P95 latence** : < 500ms maintenu sous charge

### Tests Phase 2b : Gateway KrakenD

```
# Test via gateway avec rate limiting
BASE_URL=http://localhost:8080 k6 run --duration 2m --vus 15
scripts/k6/signup.js

# Comparaison automatique 3 modes
./tests/scripts/test-krakend-gateway.sh
```

#### Résultats attendus Phase 2b :

- **Overhead gateway** : +2-5ms latence P95 vs direct
- **Rate limiting** : 429 Too Many Requests si > 100 req/s
- **Circuit breaker** : 503 Service Unavailable après 5 erreurs consécutives

## 3.4 Commandes de maintenance

```
# Redémarrage services
docker compose restart api nginx redis
```

```
# Scaling manuel (Phase 2a)
docker compose up -d --scale api=4

# Nettoyage complet
docker compose down -v && docker system prune -f

# Accès direct bases de données
docker exec -it brokerx-mysql mysql -uroot -proot123 brokerx
docker exec -it brokerx-redis redis-cli

# Backup base de données
docker exec brokerx-mysql mysqldump -uroot -proot123 brokerx >
backup-$(date +%Y%m%d).sql
```

### 3.5 Débogage problèmes courants

**Problème** : API ne démarre pas

```
# Diagnostic
docker compose logs api --tail 20
docker compose ps

# Solution
docker compose down && docker compose up -d --build
```

**Problème** : Redis déconnecté

```
# Vérifier autres Redis en conflit
docker ps | grep redis

# Stopper conflits
docker stop $(docker ps -q --filter "name=redis" --filter "name!=brokerx")

# Rebuild API avec nouvelles dépendances
docker compose build api && docker compose up -d api
```

**Problème** : Gateway 502 Bad Gateway

```
# Vérifier targets upstream
curl http://localhost:9090/api/v1/targets | grep "brokerx"

# Redémarrer gateway + load balancer
docker compose restart krakend nginx
```

**Problème** : Tests performance instables

```
# Nettoyer métriques précédentes
curl -X POST http://localhost:9090/api/v1/admin/tsdb/delete_series?match[]={job="brokerx-api"}

# Réchauffer cache avant tests
./tests/scripts/warmup-cache.sh
```

3.6 Architecture réseau (Phase 2)

Flux de requête complet :

```
Client HTTP
→ KrakenD Gateway :8080 (auth, rate limit, circuit breaker)
→ NGINX Load Balancer :8090 (round-robin)
  → [API-1:8080, API-2:8080, API-3:8080, API-4:8080]
    → Redis :6379 (cache L1: portfolios, market-data)
    → MySQL :3307 (persistence: 5 schémas logiques)
```

Ports réseau utilisés :

| Service       | Port externe | Port interne | Rôle               |
|---------------|--------------|--------------|--------------------|
| API instances | 5001-5004    | 8080         | Backend REST       |
| KrakenD       | 8080         | 8080         | Gateway public     |
| NGINX         | 8090         | 80           | Load balancer      |
| MySQL         | 3307         | 3306         | Base données       |
| Redis         | 6379         | 6379         | Cache distribué    |
| Prometheus    | 9090         | 9090         | Collecte métriques |
| Grafana       | 3000         | 3000         | Dashboards         |

4. Guide d'utilisation

**Public cible** : Utilisateurs finaux, testeurs, démonstrations client

4.1 Inscription et activation de compte (UC-01)

Interface Web

- 1. **Accéder** à l'application : http://localhost:5000
- 2. **Cliquer** sur "S'inscrire"
- 3. **Remplir** le formulaire :
  - Email (unique)
  - Nom complet

- Mot de passe (min. 8 caractères)
- Confirmer mot de passe

4. **Soumettre** → Un email OTP est envoyé

5. **Consulter** les logs pour récupérer le code OTP :

```
docker logs brokerx-api 2>&1 | grep "\[OTP\]" | tail -1
```

6. **Saisir** le code OTP dans l'interface

7. **Validation KYC** automatique (simulée)

8. **Compte activé** → Redirection vers l'authentification

## Via API REST

```
# 1. Inscription
curl -X POST http://localhost:5000/api/signup \
-H "Content-Type: application/json" \
-d '{
  "email": "test@example.com",
  "fullName": "Test User",
  "password": "SecureP@ss123",
  "confirmPassword": "SecureP@ss123"
}'

# Réponse : {"clientId": "...", "message": "OTP sent"}

# 2. Récupérer OTP depuis logs
OTP_CODE=$(docker logs brokerx-api 2>&1 | grep "\[OTP\]" | tail -1 | grep -
oP 'code=\K\d+')

# 3. Vérification OTP
curl -X POST http://localhost:5000/api/verify-otp \
-H "Content-Type: application/json" \
-d '{
  "clientId": "'$CLIENT_ID'",
  "otpCode": "'$OTP_CODE'"
}'

# Réponse : {"status": "verified", "kycStatus": "approved"}
```

## 4.2 Authentification sécurisée (UC-02)

### Interface Web

1. **Saisir** email et mot de passe
2. **MFA automatique** : Défi OTP envoyé par email
3. **Consulter logs** pour le code MFA
4. **Saisir** le code MFA

## 5. Session ouverte → Accès au tableau de bord

### Via API REST

```
# 1. Login initial
curl -X POST http://localhost:5000/api/auth/login \
-H "Content-Type: application/json" \
-d '{
  "email": "test@example.com",
  "password": "SecureP@ss123"
}'

# Réponse : {"challengeId": "...", "mfaRequired": true}

# 2. Récupérer code MFA
MFA_CODE=$(docker logs brokerx-api 2>&1 | grep "\[MFA\]" | tail -1 | grep -oP 'code=\K\d+')

# 3. Compléter MFA
curl -X POST http://localhost:5000/api/auth/mfa \
-H "Content-Type: application/json" \
-d '{
  "challengeId": "'$CHALLENGE_ID'",
  "otpCode": "'$MFA_CODE'"
}'

# Réponse : {"sessionToken": "...", "expiresAt": "..."}

```

## 4.3 Dépôt de fonds (UC-03)

### Interface Web

1. **Naviguer** vers "Mon Portefeuille"
2. **Cliquer** "Effectuer un dépôt"
3. **Saisir** le montant (ex: 1000.00)
4. **Clé d'idempotence** (optionnelle, auto-générée si vide)
5. **Confirmer** → Statut "En attente"
6. **Attendre** traitement asynchrone (~5-10 secondes)
7. **Actualiser** → Solde mis à jour

### Via API REST

```
# 1. Effectuer dépôt
curl -X POST http://localhost:5000/api/deposits \
-H "Content-Type: application/json" \
-H "Authorization: Bearer $SESSION_TOKEN" \
-d '{
  "amount": 1000.00,

```

```

    "currency": "CAD",
    "idempotencyKey": "deposit-'$(date +%s)'"
  }'

# Réponse : {"paymentTxId": "...", "status": "Pending"}

# 2. Vérifier statut (polling)
curl -X GET http://localhost:5000/api/deposits/$PAYMENT_TX_ID \
  -H "Authorization: Bearer $SESSION_TOKEN"

# Statuts possibles : Pending → Processing → Settled

# 3. Consulter solde portefeuille
curl -X GET http://localhost:5000/api/portfolio \
  -H "Authorization: Bearer $SESSION_TOKEN"

```

#### 4.4 Abonnement données de marché (UC-04) - WebSocket

##### Test WebSocket via navigateur

```

// Console navigateur sur http://localhost:5000
const ws = new WebSocket('ws://localhost:5000/ws/marketdata');

ws.onopen = function() {
  console.log('WebSocket connecté');

  // S'abonner aux symboles
  ws.send(JSON.stringify({
    action: 'subscribe',
    symbols: ['AAPL', 'GOOGL', 'TSLA']
  }));
};

ws.onmessage = function(event) {
  const quote = JSON.parse(event.data);
  console.log(`${quote.symbol}: ${quote.last} (${quote.timestamp})`);
};

// Limitation: max 100 messages/seconde par connexion

```

##### Test avec outil externe (websocat)

```

# Installation websocat
sudo apt install websocat # ou brew install websocat

# Connexion et abonnement
echo '{"action":"subscribe","symbols":["BTCUSD","ETHUSD"]}' | \
websocat ws://localhost:5000/ws/marketdata

```

```
# Flux continu de quotes simulées
#
{"symbol": "BTCUSD", "bid": 45000, "ask": 45100, "last": 45050, "timestamp": "..."}

```

## 4.5 Placement d'ordres (UC-05)

### Via API REST

```
# 1. Placer ordre MARKET (exécution immédiate)
curl -X POST http://localhost:5000/api/orders \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $SESSION_TOKEN" \
  -d '{
    "clientOrderId": "order-'$(date +%s)'",
    "symbol": "AAPL",
    "side": "BUY",
    "type": "MARKET",
    "quantity": 10
  }'

# Réponse : {"orderId": "...", "status": "FILLED", "executions": [...]}

# 2. Placer ordre LIMIT (en attente si prix pas atteint)
curl -X POST http://localhost:5000/api/orders \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $SESSION_TOKEN" \
  -d '{
    "clientOrderId": "limit-order-'$(date +%s)'",
    "symbol": "GOOGL",
    "side": "SELL",
    "type": "LIMIT",
    "quantity": 5,
    "price": 2800.00,
    "timeInForce": "GTC"
  }'

# 3. Consulter statut ordre
curl -X GET http://localhost:5000/api/orders/$ORDER_ID \
  -H "Authorization: Bearer $SESSION_TOKEN"

# 4. Annuler ordre (si pas encore exécuté)
curl -X DELETE http://localhost:5000/api/orders/$ORDER_ID \
  -H "Authorization: Bearer $SESSION_TOKEN"

# 5. Consulter positions P&L
curl -X GET http://localhost:5000/api/positions \
  -H "Authorization: Bearer $SESSION_TOKEN"

```

## Contrôles pré-trade automatiques (8 validations)

1. **Client actif** : Statut != SUSPENDED
2. **Compte actif** : Statut == ACTIVE
3. **Idempotence** : clientOrderId unique par compte
4. **Prix/quantité positifs** : > 0
5. **Solde suffisant** : Pour ordres BUY
6. **KYC valide** : Niveau >= BASIC
7. **Heures de trading** : 9h30-16h EST (simulé: toujours ouvert)
8. **Limites de risque** : Position max par symbole (simulé: 1000 actions)

## 4.6 Tableaux de bord et rapports

### Grafana - Métriques temps réel

**URL** : <http://localhost:3000> (admin/admin)

#### Dashboards disponibles :

- **API Performance** : Latence P95/P50, taux d'erreur, RPS
- **Infrastructure** : CPU, mémoire, connexions DB
- **Business Metrics** : Signups/jour, dépôts/jour, volume trading
- **Cache Redis** : Hit rate, latence, evictions

### Audit trail et logs métier

```
# Logs d'audit par UC
docker compose logs api | grep "AUDIT.*SIGNUP"
docker compose logs api | grep "AUDIT.*AUTH"
docker compose logs api | grep "AUDIT.*DEPOSIT"
docker compose logs api | grep "AUDIT.*ORDER"

# Métriques business sur période
curl "http://localhost:9090/api/v1/query_range?
query=signup_total&start=$(date -d '1 hour ago' +%s)&end=$(date
+%s)&step=60"
```

## 4.7 Cas d'erreurs et récupération

### Erreurs courantes utilisateur

- **400 Bad Request** : Données invalides (email déjà pris, mot de passe faible)
- **401 Unauthorized** : Session expirée, JWT invalide
- **409 Conflict** : Idempotence (même idempotencyKey/clientOrderId)
- **422 Unprocessable Entity** : Échec contrôles pré-trade
- **429 Too Many Requests** : Rate limiting (> 100 req/s via Gateway)

### Messages d'erreur business

```
// Exemple: Ordre rejeté - solde insuffisant
{
  "error": "Insufficient funds",
  "details": {
    "required": 15000.00,
    "available": 8500.00,
    "currency": "CAD"
  },
  "errorCode": "INSUFFICIENT_BALANCE"
}
```

## Récupération automatique

- **Reconnexion WebSocket** : Auto-reconnect après 5s de déconnexion
- **Retry idempotent** : Même clé → même résultat (dépôts, ordres)
- **Circuit breaker** : Gateway passe en mode dégradé si > 5 erreurs/minute

---

## 5. Références et bibliographie

### 5.1 Technologies et frameworks

#### Plateforme de développement :

- .NET 9.0 : <https://docs.microsoft.com/en-us/dotnet/>
- C# 12.0 : <https://docs.microsoft.com/en-us/dotnet/csharp/>
- Entity Framework Core 8.0 : <https://docs.microsoft.com/en-us/ef/core/>

#### Infrastructure et déploiement :

- Docker & Docker Compose : <https://docs.docker.com/>
- MySQL 8.0 : <https://dev.mysql.com/doc/>
- Redis 7.0 : <https://redis.io/documentation>
- NGINX 1.24 : <https://nginx.org/en/docs/>
- KrakenD 2.5 : <https://www.krakend.io/docs/>

#### Observabilité et monitoring :

- Prometheus 2.47 : <https://prometheus.io/docs/>
- Grafana 10.2 : <https://grafana.com/docs/>
- Serilog : <https://serilog.net/>

#### Tests et performance :

- k6 : <https://k6.io/docs/>
- xUnit.net : <https://xunit.net/>
- BCrypt.Net : <https://github.com/BcryptNet/bcrypt.net>

### 5.2 Patterns et principes architecturaux

**Architecture hexagonale :**

- Cockburn, A. "Hexagonal Architecture" : <https://alistair.cockburn.us/hexagonal-architecture/>
- Martin Fowler "Ports and Adapters" : <https://martinfowler.com/articles/injection.html>
- Microsoft .NET Architecture Guides : <https://docs.microsoft.com/en-us/dotnet/architecture/>

**Domain-Driven Design :**

- Domain-Driven Design Community : <https://dddcommunity.org/>
- Martin Fowler "Domain-Driven Design" : <https://martinfowler.com/tags/domain%20driven%20design.html>
- Microsoft DDD Guide : <https://docs.microsoft.com/en-us/dotnet/architecture/microservices/microservice-ddd-cqrs-patterns/>

**Microservices et API Gateway :**

- Microsoft Microservices Guide : <https://docs.microsoft.com/en-us/dotnet/architecture/microservices/>
- Martin Fowler "Microservices" : <https://martinfowler.com/articles/microservices.html>
- API Gateway Pattern : <https://microservices.io/patterns/apigateway.html>

## 5.3 Standards et bonnes pratiques

**APIs REST :**

- REST API Tutorial : <https://restfulapi.net/>
- Mozilla HTTP Documentation : <https://developer.mozilla.org/en-US/docs/Web/HTTP>
- Microsoft REST API Guidelines : <https://github.com/Microsoft/api-guidelines>

**Sécurité :**

- JWT.io - JSON Web Tokens : <https://jwt.io/>
- OWASP API Security Top 10 : <https://owasp.org/www-project-api-security/>
- Auth0 Security Best Practices : <https://auth0.com/docs/secure>

**Observabilité et monitoring :**

- Google SRE Book (4 Golden Signals) : <https://sre.google/sre-book/monitoring-distributed-systems/>
- OpenTelemetry Documentation : <https://opentelemetry.io/docs/>
- Prometheus Best Practices : <https://prometheus.io/docs/practices/>

## 5.4 Documentation du projet

**Architecture Decision Records (ADR) :**

- ADR-001 : Architecture hexagonale (Ports & Adapters)
- ADR-002 : Persistance et Idempotence (EF Core, Ledger append-only)
- ADR-003 : Stratégie d'erreurs et versionnage API
- ADR-004 : NGINX Load Balancer (Phase 2a)
- ADR-005 : KrakenD Gateway (Phase 2b)
- ADR-006 : Redis Cache distribué

**Diagrammes 4+1 :**

- Vue Logique : Domaine métier, bounded contexts, ports
- Vue Processus : Flux UC, observabilité, communication
- Vue Développement : Structure projets .NET, dépendances
- Vue Déploiement : Évolution monolithe → microservices
- Scénarios : UC-01 à UC-05 détaillés

## 5.5 Outils de développement

### **IDE et productivité :**

- Visual Studio Code : <https://code.visualstudio.com/>
- PlantUML : <https://plantuml.com/>
- Postman : <https://www.postman.com/>

### **CI/CD et DevOps :**

- GitHub Actions : <https://docs.github.com/en/actions>
  - Docker Hub : <https://hub.docker.com/>
-